Secure Multiparty Computation in Finance: Private Set Intersection

Guilherme Marques Alves
Luís Silva

February 8, 2026

1 Introduction

In this project, we will explore the principles **Secure Multiparty Computation** (SMC), with a focus on **Private Set Intersection** (PSI). We will evaluate and compare each of the 4 protocols with respect to the execution time and total exchanged data.

1.1 Theory Overview

PSI is a central component of Secure Multiparty Computation, designed to solve the problem of finding $A \cap B$ while preserving privacy between A and B. For this purpose, we briefly review the protocols evaluated:

1.1.1 Naive Hashing

The **Naive Hashing** protocol represents the simplest and most basic approach to finding a set intersection ($A \cap B$). While it is highly efficient in terms of computational speed and data (it uses simple hash operations), it fails to meet the strict privacy requirements of Secure Multiparty Computation (SMC).

The mechanism involves only two steps:

1. **Element Hashing and Exchange:**
   - Party A calculates a hash function (H) for every element in its private set: $H(A_i)$.
   - Party B calculates a hash function (H) for every element in its private set: $H(B_j)$.
   - Both parties then exchange their entire lists of hashes over the communication channel.
2. **Intersection Determination:**
   - Both Party A and Party B compare the list of hashes received from the other party against their own local list of hashes.
   - Any hash that matches in both lists corresponds to an element of the intersection ($A \cap B$).
3. **Security Risk**: The major drawback is that the protocol is vulnerable to dictionary and brute-force attacks.

1.1.2 Server-Aided

The **Server-Aided** protocol delegates the computationally intensive task of finding the set intersection to a third party, known as the Server. This method is conceptually simple and often results in high performance, but introduces a major trust assumption.

1. **Roles:** There are three parties involved: Party A, Party B, and the Server (S).
2. **Setup:** Parties A and B agree on a method to obfuscate (hide) their data, typically using encryption or a cryptographic blinding technique, before sending it to S.
3. **Data Transfer:**
   - Party A sends its obfuscated set A' to S.
   - Party B sends its obfuscated set B' to S.
4. **Intersection Calculation:** The Server S performs the intersection operation on A' and B' to determine $A' \cap B'$. Because S receives the data in an obfuscated form, S theoretically cannot learn the content of the elements themselves, but can learn the intersection result.

5. **Result Distribution:** The Server S returns the intersection set $A' \cap B'$ to both Party A and Party B. These parties can then decrypt or de-obfuscate the result to retrieve the actual common elements, $A \cap B$.

6. **Security Trade-off:** The primary drawback of this method is the requirement for complete trust in the Server S. If the Server is malicious or compromised, the privacy of both Party A's and Party B's entire sets may be compromised, failing the core security premise of SMC.

1.1.3 Diffie-Hellman-based

This PSI protocol leverages the homomorphic properties of modular exponentiation based on the Diffie-Hellman (DH) key exchange protocol to derive common blinded values for the intersection.

1. **Setup:** The parties agree on public parameters (p, g) and choose private secrets a (Party A) and b (Party B).

2. **Element Blinding:** For each element x_i in their respective set, each party computes a blinded version of the element using their secret. The element is first transformed into a suitable representation (e.g., using a *hash* function H):

   - Party A computes $(H(x_i))^a \pmod p$.
   - Party B computes $(H(x_j))^b \pmod p$.

3. **Exchange and Finalization:** The parties exchange their lists of blinded values. Each party then applies the other party's secret (via their own secret) to the received values.

   - Party A computes $Final_A = ((H(x_j))^b)^a = H(x_j)^{ab} \pmod p$.
   - Party B computes $Final_B = ((H(x_i))^a)^b = H(x_i)^{ab} \pmod p$.

4. **Intersection:** If $x_i = x_j$, then $Final_A$ will equal $Final_B$. The intersection is found by comparing the final lists of blinded values, without either party learning elements of the other party not present in the intersection.

1.1.4 OT-based

The **Oblivious Transfer (OT)** ensures that information is transferred such that neither party learns anything beyond the required intersection.

The most common variant used for PSI is the 1-out-of-N OT, typically implemented as 1-out-of-2 OT (OT_1^2), which works as follows:

1. **Setup:** The Sender (Party A) holds two secret messages (M_0, M_1). The Receiver (Party B) holds a selection bit ($b \in \{0, 1\}$) indicating which message they wish to receive (M_b).

2. **Transfer Mechanism:** A cryptographic mechanism is established. The core security lies in two properties:

   - **Receiver Obliviousness**: Party B successfully receives M_b, but learns nothing about the other message (M_{1-b}).
   - **Sender Obliviousness**: Party A does not learn the selection bit b, meaning they do not know which message Party B received.

3. **OT in PSI:** To achieve PSI, the OT protocol is run repeatedly. Party A acts as the Sender, and Party B acts as the Receiver. The messages (M_i) are typically cryptographically blinded or encrypted representations of Party A's set elements, and the selection bits determine which of A's elements match B's elements.

4. **Security Guarantee:** The OT-based protocol is highly secure because it satisfies the semi-honest security model, ensuring maximal privacy by design, which is essential for handling sensitive data like client IDs.

1.2 Problem

For this project we defined the following problems and why PSI can be the solution:

- **Marketing**: If two financial institutes plan a collaborative project offering special conditions to clients of both institutes;

- **Market Analysis**: If two financial institutes plan to fuse, they need to assess the overlap in their High-Net-Worth Individual (HNWI) client base to estimate the market size or optimize joint portfolio strategies.

- **Fraud Detection**: Identifing the same client in different financial institutes of the same field can indicate fraud;

Those three problems can be solved using PSI. PSI will show the intersections while not disclosing the whole client list.

1.3 Choosing the dataset

To guarantee rigorous and reproducable tests, we decided to create a syntetic dataset using a Python script:

numbers

```python
import pandas as pd
import numpy as np
import os
from tqdm import tqdm

DATASET_SIZES = [5000, 10000, 25000, 50000, 100000]
INTERSECTION_RATE = 0.10
ID_MIN_VALUE = 100000000
ID_MAX_VALUE = 999999999
COLUMN_NAME = "ClientID"
OUTPUT_DIR = "psi_quant_datasets"

def create_dataset_version(size, rate, path):
    intersection_size = int(size * rate)
    unique_size = size - intersection_size
    total_unique_ids_needed = intersection_size + unique_size + unique_size
    id_range_available = ID_MAX_VALUE - ID_MIN_VALUE + 1
    if total_unique_ids_needed > id_range_available:
        return

    rng = np.random.default_rng()
    try:
        id_pool = rng.choice(
            np.arange(ID_MIN_VALUE, ID_MAX_VALUE + 1),
            size=total_unique_ids_needed,
            replace=False
        )
    except ValueError as e:
        return

    common_ids = id_pool[0:intersection_size]
    unique_a = id_pool[intersection_size : intersection_size + unique_size]
    unique_b = id_pool[intersection_size + unique_size : total_unique_ids_needed]
    party_a_ids = np.concatenate([common_ids, unique_a])
    party_b_ids = np.concatenate([common_ids, unique_b])

    rng.shuffle(party_a_ids)
    rng.shuffle(party_b_ids)

    df_a = pd.DataFrame(party_a_ids, columns=[COLUMN_NAME])
    df_b = pd.DataFrame(party_b_ids, columns=[COLUMN_NAME])

    df_a.to_csv(os.path.join(path, "party_A.csv"), index=False)
    df_b.to_csv(os.path.join(path, "party_B.csv"), index=False)


def main():
    if not os.path.exists(OUTPUT_DIR):
        os.makedirs(OUTPUT_DIR)

    for size in tqdm(DATASET_SIZES):
        version_path = os.path.join(OUTPUT_DIR, f"dataset_{size}")
        if not os.path.exists(version_path):
            os.makedirs(version_path)

        create_dataset_version(size, INTERSECTION_RATE, version_path)

        intersection_actual = int(size * INTERSECTION_RATE)

if __name__ == "__main__":
    main()
```

So the objective of PSI in this dataset is to find the intersections of **ClientID**, this is adequate for PSI because:

- PSI is a protocol designed to operate on discrete sets, since ClientID is a 9 digit int alongside being a primary key, guarantees the unicity;
- Since ClientID is highly sensitive in the context of finance, PSI is adequate because it allows privacy;
- Since ClientID is an int, it optimizes the conversion to hash, resulting in a higher effiency on PSI comparing to other types of data.

This script will also:

- Guarantee an exact intersection of 10% between parties (this is great to benchmark different protocols);
- Create datasets of 5000, 10000, 25000, 50000, 100000 records.

2 Benchmarking

2.1 Predictions

2.1.1 Naive Hashing

Since this method is dominated by hashing operations (which are very fast and efficient) we predict that this protocol will be the fastest and the cheaper.

But this method is vulnerable to dictionary and brute force attacks.

2.1.2 Server-Aided

- Execution time is expected to be very fast, comparable to Naive Hashing;
- Total Data Exchange will be low, since it only requires to send the sets of both parties to the sever;
- It is not secure, it requires on the complete trust of the external server.

2.1.3 Diffie-Helman

- We predict this protocol will be the slowest of all. It needs to perform modular exponentiation with very large prime numbers, which is a computationally expensive operation;
- Total data exchange will be high. It requires large cryptographic public keys;
- It offers robust security based on the difficulty of the Discrete Logarithm.

2.1.4 Oblivous Tranfering - based

- We expect the execution time to be slower than Naive Hashing by faster than Diffie-Hellman-based;
- We predict a high data exchange, compared to Diffie-Hellman-based. It requires exchanging multiple pairs of encrypted values to ensure the obliviousness ;
- It offers a strong privacy guarantee.

2.2 Results

Table 1: Results from 5 simulations in each row

Protocol	Dataset Size (n)	μ_{Time} (s)	σ_{Time} (s)	$\mu_{\text{Total Data}}$ (kB)	$\sigma_{\text{Total Data}}$ (kB)
Naive Hashing	5000	0.019	0.002	90.4	0.894
	10000	0.025	0.003	182.0	0.000
	25000	0.047	0.012	452.0	0.000
	50000	0.090	0.015	904.0	0.000
	100000	0.110	0.027	1947.0	57.814
Server-aided	5000	0.180	0.034	166.0	0.000
	10000	0.118	0.056	327.2	1.304
	25000	0.133	0.049	810.6	2.074
	50000	0.255	0.037	1619.7	1.633
	100000	0.497	0.132	2557.0	6.403
Diffie-Helman	5000	3.682	0.059	558.4	27.610
	10000	7.177	0.178	1083.8	18.075
	25000	17.434	0.093	2582.6	163.344
	50000	34.487	0.144	4544.2	485.524
	100000	68.537	0.245	7434.2	1066.057
OD-based	5000	0.339	0.043	549.6	0.548
	10000	0.351	0.029	1080.0	0.707
	25000	0.398	0.004	2634.8	0.447
	50000	0.487	0.018	4670.8	653.893
	100000	0.658	0.027	8785.0	1266.402

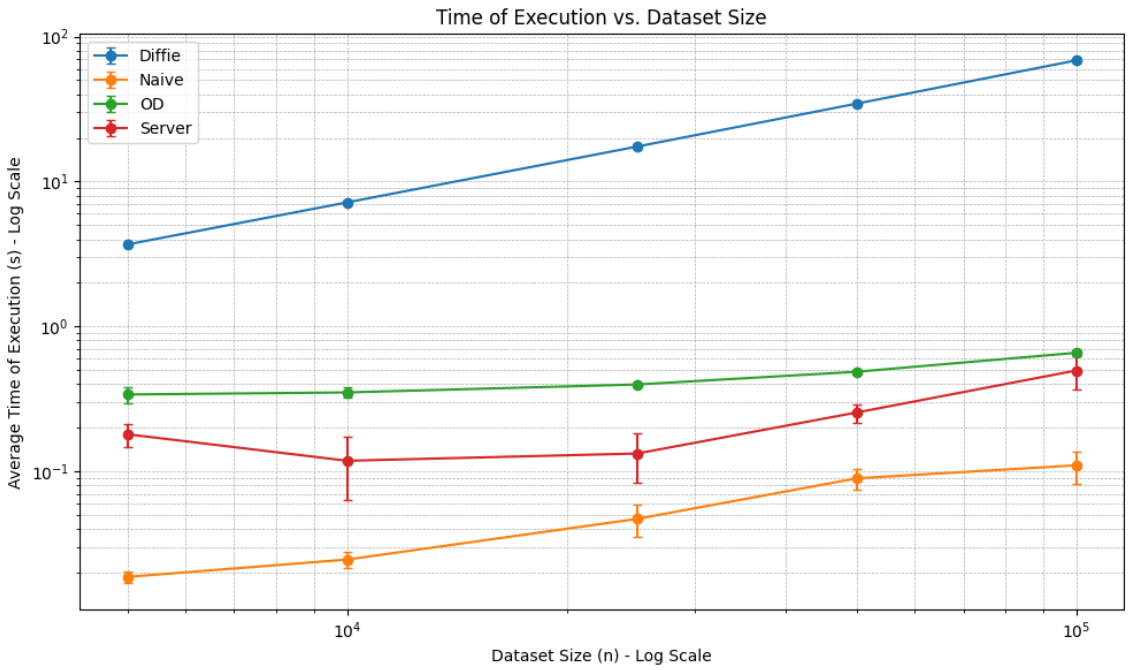

Figure 1: Time of Execution versus Dataset Size

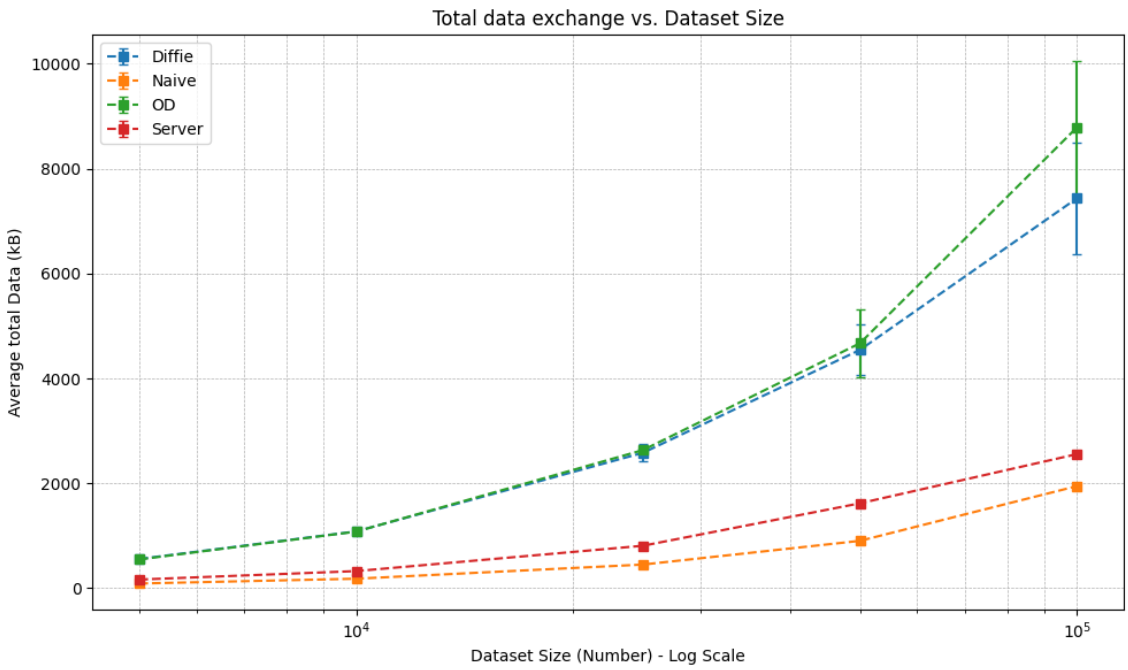

Figure 2: Total Data Exchanged versus Dataset Size

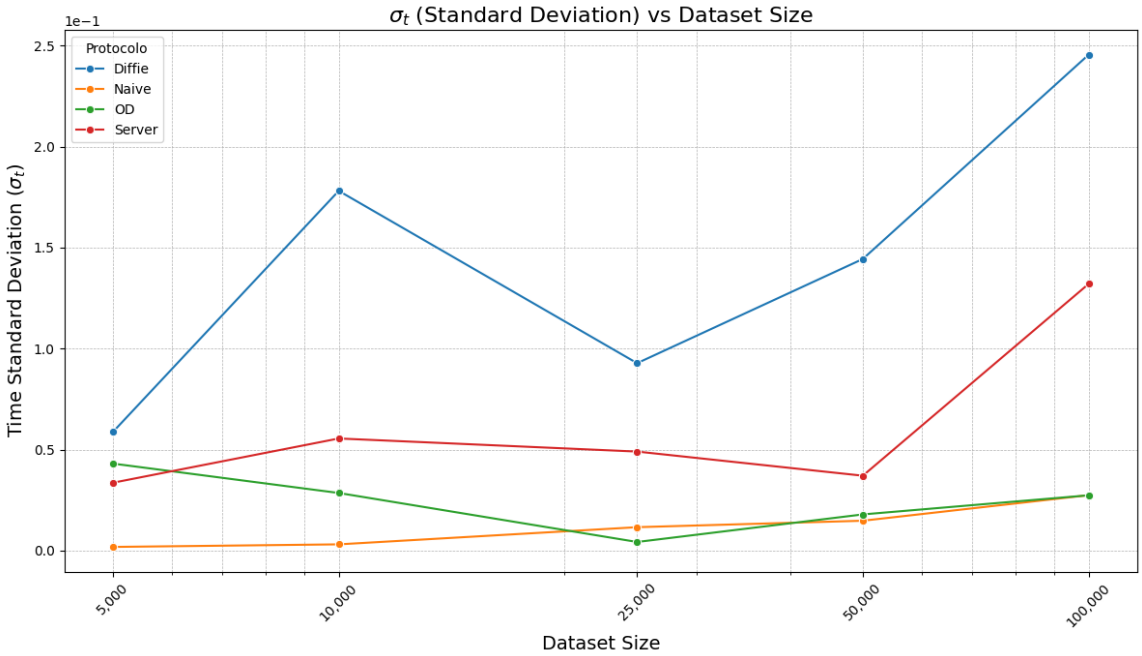

Figure 3: Time Standard Deviation σ_t versus Dataset Size

Log scale was decided on Average Time of Execution for visual purposes.

2.3 Discussion

2.3.1 Time of Execution

The analysis of the execution time (Figure 1) reveals significant differences in performance and computational complexity across the protocols:

- **Naive Hashing**: This is the fastest protocol, which is due to the computational simplicity of the dominant hashing operations. However, it is fundamentally vulnerable to dictionary and brute force attacks.

- **Server-aided**: This is the second fastest protocol. Despite its speed, it is not recommended for our problems since it requires a fully trusted third party, thus lacking the necessary privacy guarantee for Secure Multiparty Computation (SMC).
- **OT-based**: This is the third fastest protocol. It is the most secure among the practical contenders and is significantly faster than Diffie-Hellman-based approaches.
- **Diffie-Hellman-based**: This is the slowest protocol, due to the high computational complexity of its modular exponential operations. The time difference is stark, reaching 68.537s for $N = 100.000$, which is approximately 100 times slower than the OT-based protocol (0.658s).

2.3.2 Total Data Exchanged

OT-based and Diffie-Hellman-based are the protocols that exchange more data, especially at $n = 100000$.

- **OT-based**: For this protocol, volume data increases exponentially with the dataset size
- **Diffie-Hellman-based**: Total data exchange is a direct consequence of the need of sending large cryptographic keys (modular exponential)

The most efficient in data volume are the protocols, **Naive Hashing** and **Server-aided**.

- **Naive Hashing**: Since this protocol is dominated by hashing operations (which are very simple), the protocol needs less volume data to send and receive;
- **Server-aided**: Despite needing a third party, the two parties only need to send their sets one single time to the server; this results in less data volume than other protocols.

2.3.3 Time Standard Deviation σ_t

The time standard deviation (σ_t), visualized in Figure 3, represents the stability and predictability of the protocols. The **Naive Hashing** protocol shows a consistently low σ_t across all dataset sizes, confirming its high reliability and minimal variance due to its computational simplicity. Conversely, the Diffie-Hellman-based protocol exhibits the highest σ_t with significant spikes (e.g., at $N = 10,000$ and $N = 100,000$), indicating that its execution time is highly volatile, likely due to the extreme computational load of modular exponentiation. The OT-based protocol maintains a relatively low and stable σ_t, proving its robust and predictable performance, which is a critical factor for real-time applications such as those in the financial domain.

3 Conclusion

OT-based approaches proved to be a compromise between security and velocity. It presents a consistently small execution time and shows a low standard deviation σ_t, indicating that this protocol is robust. It is essential to note that this protocol overcarries total volume, which makes it difficult with large datasets. For those reasons, we conclude that for our proposed problems, OT-based is the best protocol to choose.